Osdag®
Open steel design and graphics

Parametric 3D CAD Model of a Steel Girder Bridge (pythonOCC)

Introduction

Osdag is a free and open-source software for the design and detailing of steel structures. As part of the bridge module development, this task evaluates your ability to:

- Develop **parametric 3D CAD models**
- Structure reusable and modular code
- Apply **engineering logic in geometry creation**
- Model **reinforced concrete components with detailing awareness**

Problem Statement:

Create a parametric, modular 3D CAD model of a short-span steel girder bridge using the `pythonOCC` (pythonocc-core) library. The model should be assembled from component factories (girders, deck slab, piers, pier cap, piles, pile cap, reinforcement bars) so each part is adjustable by changing parameters. The final deliverable is a runnable `bridge_model.py` script that builds, visualizes, and optionally exports the bridge geometry. Concrete parts should be semi-transparent so internal steel reinforcement bars are visible.

Objective

- Implement a reusable, parameter-driven script (`bridge_model.py`) that assembles a complete short-span bridge model from smaller component factories.
- Deliver visual output (OCC display), tests, documentation, and packaged submission files.

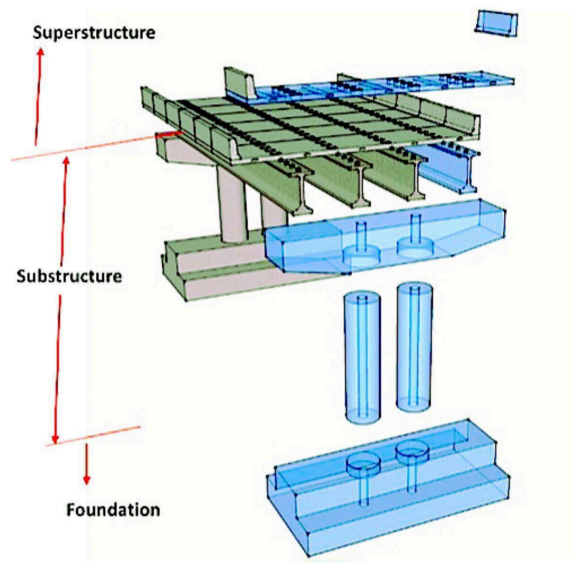
Context

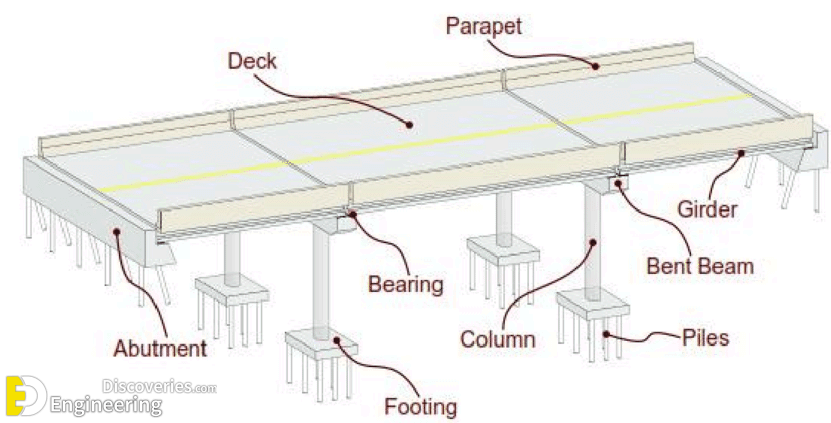
Short-span steel girder bridges are widely used on highways and local roads. This exercise focuses on modeling a single span bridge with:

- Main longitudinal steel girders (I-sections) — three or more
- Concrete deck slab (rectangular prism segments)
- Concrete piers (circular columns)
- Pier cap (hammerhead / trapezoidal cap)
- Piles/ Columns (circular)
- Pile cap (rectangular)
- Steel reinforcement bars inside concrete parts (visible through semi-transparent concrete)
- Parapets/railings (optional)

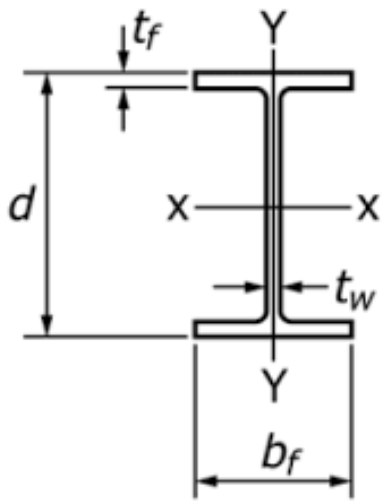
Pictures are attached to understand the bridge parts, expected output

1. **Bridge Components**

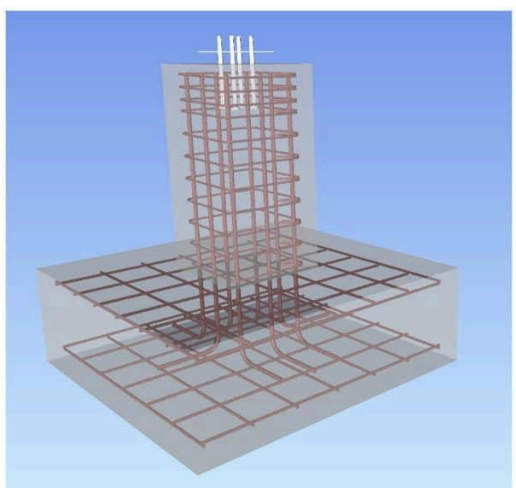




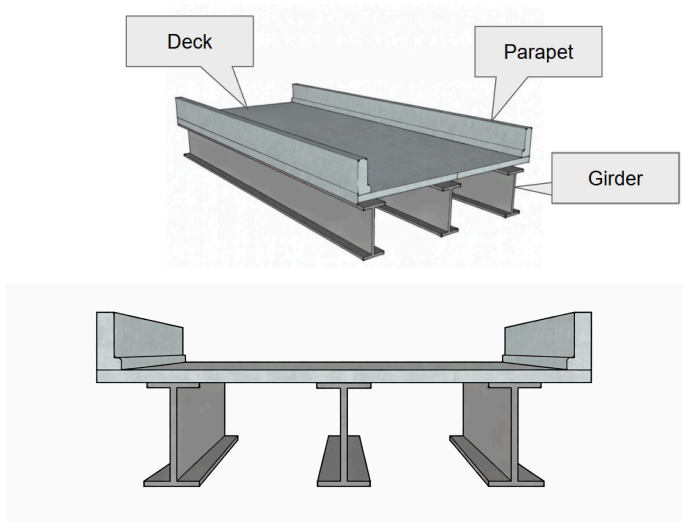
2. **Main girder section (I-section dims: d, bf, t**f, tw)



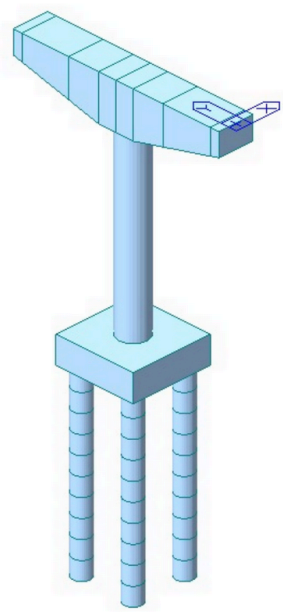
3. **Steel reinforcement bars in concrete parts**



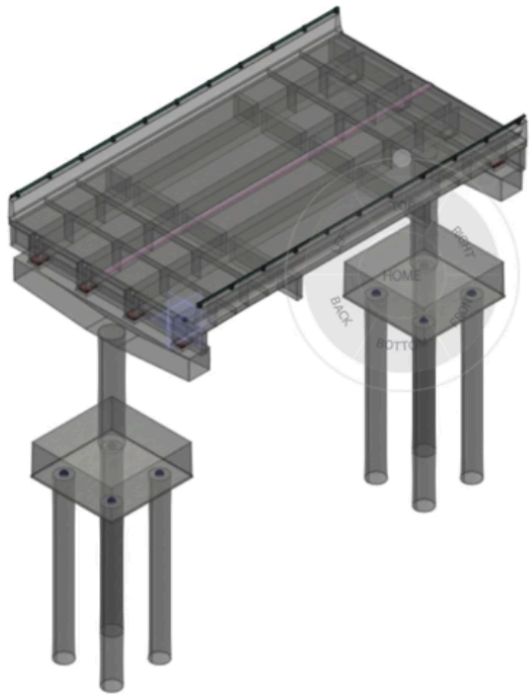
4. **Bridge Superstructure (Required to model)**



5. **Bridge Substructure with foundation (Required to model)**



6. **Expected final output (Fuse above 2)**



Coordinate System

To ensure consistency:

- X-axis → Longitudinal direction (span)
- Y-axis → Transverse direction (deck width)
- Z-axis → Vertical direction
- **Origin:** Center of span at deck level (or clearly defined and documented)

List of Files

Here is a list of the OCC files provided for the problem:

- **`draw_i_section.py`**: Download Contains a function to create a 3D model of an I-section beam.
- **`draw_rectangular_prism.py`**: Download Contains a function to create a 3D model of a rectangular prism

Parametric Design (parameters to expose at the top of `bridge_model.py`)

Make all dimensions and counts adjustable through named variables (and optionally CLI flags). The list below describes the required parameters and suggested additional parameters to expose.

1. Geometry & layout

- `units` — string, `"mm"` or `"m"` (document choice)
- `span_length_L` — total clear span length (between pier faces), e.g., `12000` mm
- `n_girders` — number of main longitudinal girders (≥ 3)
- `girder_centroid_spacing` — centre-to-centre spacing between adjacent main girders
- `girder_offset_from_edge` — overhang distance from outermost girder to deck edge (half the deck overhang)
- `n_crossframes` or `crossframe_spacing` — spacing or count of cross-frames along span
- `deck_width` — total deck width (across girders plus overhangs)
- `n_lanes` and `lane_width` — optional for demarcation/placement
- `deck_overhang` — deck overhang beyond outer girders

2. Main girder (I-section) parameters

- `girder_section_d` — depth of I-section (d)
- `girder_section_bf` — flange width (bf)
- `girder_section_tf` — flange thickness (tf)
- `girder_section_tw` — web thickness (tw)
- `girder_length` — equal to `span_length_L` ± end conditions
- `girder_material` — optional material name

3. Deck slab

- `deck_thickness`
- `deck_slab_segment_length` — optional segmentation length for modelling as many prisms
- `deck_cover` — concrete cover for rebar
- `deck_material_opacity` — set semi-transparent alpha (e.g., `0.5`)

4. Pier & pier cap

- `pier_diameter` — circular pier diameter
- `pier_height` — from pile cap top to pier cap bottom (or ground to deck)
- `pier_location` — coordinate along span (typically at span centerline)

- `pier_cap_length`, `pier_cap_width`, `pier_cap_depth` — trapezoidal / hammerhead dimensions
- `pier_cap_elevation` — vertical location relative to pier top

5. Pile & pile cap

- `n_piles_per_cap` — number of piles under each pile cap (arranged grid)
- `pile_diameter`
- `pile_length`
- `pile_spacing` — spacing between pile centrelines
- `pile_cap_length`, `pile_cap_width`, `pile_cap_depth` — rectangular pile cap dims
- `pile_cap_elevation` — vertical location of top of pile cap relative to ground

6. Reinforcement (visible inside concrete)

- `rebar_main_diameter` — diameter of longitudinal reinforcement (e.g., 16 mm)
- `rebar_transverse_diameter` — diameter of ties/stirrups (e.g., 8 mm)
- `rebar_spacing_longitudinal` — spacing of longitudinal bars (mm)
- `rebar_spacing_transverse` — spacing for ties (mm)
- `rebar_cover` — concrete cover (mm)
- `rebar_visible` — boolean to toggle rebar display
- `concrete_opacity` — e.g., `0.3` to show semi-transparent concrete

7. Visualization & output

- `show_axes` — boolean
- `background_color` — e.g., white or grey
- `save_step` — boolean to export STEP file
- `step_filename`
- `save_brep` — boolean + filename
- `render_size` — optional screenshot resolution

Functional Requirements (what the script must implement)

A. **Component factories**: Use or implement factory functions that return TopoDS solids:

a. `create_i_section(d, bf, tf, tw, length)` — returns I-section solid extruded to length (use existing `draw_i_section.py`)
b. `create_rectangular_prism(width, height, length)` — returns prism (use existing `draw_rectangular_prism.py`)
c. `create_circular_pier(diameter, height)` — helper to create a circular column
d. `create_trapezoidal_pier_cap(length, width_top, width_bottom, depth)` — creates a hammerhead cap
e. `create_pile(diameter, length)` — circular pile solid
f. `create_pile_cap(length, width, depth)` — rectangular prism
g. `create_rebar_grid_for_deck(deck_width, span_length, cover, diam, spacing)` — returns as cylindrical bars placed inside deck volume

B. **Assembly functions**:

a. `build_girders()` — create and position `n_girders` along deck cross-section
b. `build_crossframes()` — create and position cross-frames at intervals along the span
c. `build_deck()` — construct deck as prism(s), Boolean fuse with openings if desired
d. `build_piers_and_pilecaps()` — create piers and pile caps at specified locations and assign piles beneath
e. `assemble_bridge()` — position all components into one assembly (compound)

C. **Parametric transforms**: use translations/rotations to place each component correctly in 3D.
D. **Concrete & rebar visibility**:

a. Concrete solids (deck, pier, pier cap, pile cap) must be displayed semi-transparent (`opacity < 1`).
b. Rebar solids (cylinders) displayed as opaque steel color; they must intersect logically with concrete geometry (placed inside volumes with correct cover).

E. **Visualization**:

a. Initialize OCC display and show each topology with appropriate color and transparency.
b. Allow toggling of rebar visibility.
c. Provide camera preset (isometric) and zoom-to-fit.

F. **Export**:

a. Option to export the whole assembly or individual parts to STEP/BREP/IFC files.

Example Dimensions (default example values — units: millimetres)

You should use these example values for demonstration and include them as defaults in the script.

- Units: `mm`
- Span length: `span_length_L = 12000`
- Number of main girders: `n_girders = 3`
- Girder spacing (center to center): `girder_centroid_spacing = 3000`
- Girder I-section (steel, example ISMB-section style):
  - `girder_section_d = 900`
  - `girder_section_bf = 300`
  - `girder_section_tf = 16`
  - `girder_section_tw = 10`
- Deck:
  - `deck_width = 7000` ((n_girders − 1) × girder_spacing + 2 × deck_overhang)
  - `deck_thickness = 200`
  - `deck_overhang = 500`
- Pier:
  - `pier_diameter = 800`
  - `pier_height = 3000`
  - `pier_cap_top_width = 3000`
  - `pier_cap_bottom_width = 1200`
  - `pier_cap_depth = 600`
- Pile & pile cap:
  - `n_piles_per_cap = 4` (2x2)
  - `pile_diameter = 400`
  - `pile_length = 5000`
  - `pile_cap_length = 2200`
  - `pile_cap_width = 1200`
  - `pile_cap_depth = 600`
  - `pile_spacing = 600`

- Reinforcement:
  - `rebar_main_diameter = 16`
  - `rebar_spacing_longitudinal = 150`
  - `rebar_transverse_diameter = 8`
  - `rebar_spacing_transverse = 200`
  - `rebar_cover = 40`

- Visualization:
  - `concrete_opacity = 0.35`
  - `rebar_visible = True`

(You may change the numeric defaults for your test cases — ensure units are consistent and documented.)

Resources

- **OCC Documentation**
- **OCC GitHub Repository**

Feel free to adjust the specifics of the example dimensions and other details as needed!

Submission Requirements:

Submit the completed `bridge_model.py` script along with other files. Ensure that your script runs without errors and that the 3D model is correctly generated and displayed.

- **Link 1: Video Demonstration**
  - Create a short video showcasing the functionality of your code. A simple screengrab is sufficient; no need for curation. The video can be silent. Upload it as an unlisted video on YouTube. Provide the link.
- **Link 2: GitHub Repository Link**
  - Provide the GitHub repository link and add `osdag-admin` as a collaborator.
- **Report: Documentation**
  - Submit a PDF document explaining the code you have created.
- **ZIP File**
  - Submit a ZIP file containing all relevant files and codes for the project.

Note: The deliverables listed are very basic requirements only. You are expected to bring your own creative ideas to the project as you wish. The files/codes provided give only a basic layout/template for your project. Please modify/enhance as you feel best.